Room Agent 技术规格说明（Spec）

> 本文档定义 **Room Agent 的最小且必要技术规格**，目标是支持可运行、可演进的家居 / 空间级 Agent 系统，而非追求过度抽象或形式化标准。
>
> 数字人Agent为实验室的Room Agent，因此适用于本文档中的需求。

---

1. 设计原则

1.1 核心原则

1. **房间是第一公民（Room-first）**
   Room Agent 以“物理空间”为中心，而不是以设备或技能为中心。
2. **状态优先于能力（State > Skill）**
   Room Agent 的核心职责是维护“房间状态”，而不是提供大量可调用接口。
3. **最小对外接口**
   只为跨 Agent 协作暴露必要能力，内部逻辑不强制 Skill 化。
4. **实现自由，接口稳定**
   Room Agent 内部实现可随时替换，但对外行为语义保持稳定。

---

2. 系统定位

2.1 定义

Room Agent 是负责对一个物理房间或功能空间进行：

- 状态聚合
- 局部决策
- 对外汇报
- 指令执行

的常驻 Agent 实例。

2.2 边界

Room Agent：

- ✅ 管理房间内设备、传感器、子 Agent

- ✅ 决定“房间当前是什么状态”
- ❌ 不做全局任务规划
- ❌ 不直接操作底层硬件驱动

---

3. 房间状态模型（核心）

3.1 Room State 对象

Room Agent **必须维护一个统一的 Room State**，作为唯一可信源。

代码块

```
{
  "room_id": "bedroom_01",
  "mode": "sleep",
  "occupancy": true,
  "environment": {
    "temperature": 25.1,
    "humidity": 60,
    "light": "dim"
  },
  "devices": {
    "light": "off",
    "curtain": "closed",
    "ac": "on"
  },
  "agents": {
    "robot": "idle",
    "sensor": "online"
  },
  "timestamp": 1730000000
}
```

3.2 状态更新来源

- 设备 Agent 上报
- 传感器数据
- 上层 Agent 指令
- Room Agent 内部推断

---

4. 行为模型（非 Skill 化）

4.1 内部行为

Room Agent 内部行为**不要求 Skill 形式**，可以是：

- 状态机
- 规则引擎
- 硬编码逻辑

示例：

```
代码块

IF mode == sleep AND occupancy == true
→ curtain = closed
→ light = off
```

4.2 行为触发来源

- Room State 变化
- 定时器
- 外部指令

---

5. 对外能力接口（极简）

> 仅定义 **跨 Agent 必需的最小接口集合**。

5.1 必选接口

5.1.1 获取房间状态

```
代码块

get_room_state()
```

- 返回当前 Room State 快照
- 只读

5.1.2 设置房间模式

```
set_room_mode(mode)
```

- mode 为有限枚举：
  - idle
  - sleep
  - meeting
  - custom
- 具体行为由 Room Agent 内部决定

5.1.3 房间可用性声明

```
get_room_capability()
```

- 返回房间可支持的能力集合（非设备级）

5.2 可选接口

- request_device_action(device, action)
- request_robot_service(type)

（是否暴露由部署决定）

---

6. Beacon 与发现机制

> Beacon 的**首要目标是服务 Personal Agent 的快速感知与决策**，而非 Agent 之间的泛发现。

6.1 Beacon 目标

- 为 Personal Agent 提供**低成本、低延迟的环境摘要**
- 支持 Personal Agent 在不建立连接的情况下完成初步决策
- 作为 Personal Agent 选择“是否进一步交互”的依据
- 用于快速发现 Room Agent

- 用于广播最小状态摘要

6.2 Beacon 内容（最小）

Beacon 内容应围绕 **“是否值得 Personal Agent 介入”** 这一问题设计。

代码块

```
{
  "agent": "room",
  "room_id": "bedroom_01",
  "mode": "sleep",
  "occupancy": true
}
```

- 不广播设备级细节
- 不广播隐私信息

---

7. 协作模型

> Room Agent 的主要服务对象并非抽象的“上层 Agent”，而是 **Personal Agent（个人 Agent）**。Room Agent 的设计应优先满足 Personal Agent 的决策需求。

7.1 Personal Agent → Room Agent

Personal Agent 是 Room Agent 的**主要消费者与决策主体**。

Personal Agent 通过 Room Agent：

- 感知当前房间状态（用于意图理解与决策）
- 判断是否适合执行某个用户意图
- 决定是否介入、切换房间或下发指令

典型交互包括：

- 查询房间状态
- 监听房间 Beacon
- 请求房间模式变更
- 查询房间状态
- 请求房间模式变更

7.2 Room Agent → 子 Agent

Room Agent 对下游子 Agent（设备 / 机器人 / 传感器 Agent）承担**执行协调者**角色。

- 将来自 Personal Agent 的高层决策转化为具体动作
- 屏蔽子 Agent 的实现细节
- 汇总执行结果并更新 Room State
- 下发具体动作指令
- 不暴露子 Agent 细节给上层

---

8. 运行与部署假设

- 运行平台：Jetson Nano / x86 Linux
- 常驻进程

---

9. 非功能性要求

- 子 Agent 掉线不导致 Room Agent 崩溃

---

10. 明确不做的事情（Anti-Spec）

Room Agent **明确不负责**：

- Skill 市场兼容
- 通用设备抽象标准
- Prompt 工程
- 多用户对话管理